

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO**



**BÁO CÁO MÔN HỌC
KỸ THUẬT VÀ CÔNG NGHỆ DỮ LIỆU LỚN**

**ĐỀ TÀI
GitHub Repository Analytics System**

Nhóm sinh viên thực hiện:

1. Lê Hồng Anh – 23020327
2. Hoàng Mạnh Hùng – 23020371
3. Đỗ Việt Dũng – 23020343

Giảng viên hướng dẫn:

TS. Trần Hồng Việt
ThS. Ngô Minh Hương

Hà Nội, 2025

Mục lục

MỞ ĐẦU	3
1 Tổng quan	3
1.1 Giới thiệu	3
1.1.1 Tại sao phân tích repositories GitHub lại quan trọng?	3
1.1.2 Mục tiêu cụ thể của dự án	4
1.1.3 Phạm vi dự án	5
1.2 Một số nghiên cứu liên quan	5
1.2.1 Tổng quan về các nghiên cứu	5
1.2.2 Các kỹ thuật phổ biến	6
1.2.3 Các đặc trưng dùng cho phân tích	6
1.2.4 Vai trò của Big Data trong phân tích GitHub	6
2 Phương pháp	7
2.1 Các công nghệ sử dụng (Big Data)	7
2.1.1 Apache Spark Streaming	7
2.1.2 Docker & Docker Compose	8
2.1.3 Redis	9
2.1.4 GitHub API (Nguồn dữ liệu streaming)	10
2.2 Thu thập và Xử lý Dữ liệu	10
2.2.1 Thu thập Dữ liệu	11
2.2.2 Xử lý Dữ liệu	13
2.2.3 Quy trình Xử lý	16
2.2.4 Tổng Quan Luồng Dữ Liệu	19
2.2.5 Các Quyết Định Thiết Kế	20
2.2.6 Độ Tin Cậy và Khả năng Chịu lỗi	21

2.2.7	Kết luận	21
2.3	Phương pháp tổng hợp và phân tích	21
2.3.1	Các Nhiệm Vụ Phân Tích Chính	22
2.3.2	Các nhiệm vụ phân tích nâng cao	27
2.4	Triển khai hệ thống	34
2.4.1	Kiến trúc tổng thể	37
2.4.2	Quy trình triển khai	38
2.5	Kết quả	43
2.5.1	Kết quả yêu cầu 1: Tổng số kho theo ngôn ngữ lập trình	43
2.5.2	Kết quả yêu cầu 2: Các lần push trong 60 giây gần nhất	44
2.5.3	Kết quả yêu cầu 3: Số sao trung bình theo ngôn ngữ	45
2.5.4	Kết quả yêu cầu 4: 10 từ xuất hiện nhiều nhất	46
2.5.5	Kết quả yêu cầu 5: Phân tích chủ đề - LDA	47
2.5.6	Kết quả yêu cầu 6: Nhận diện thực thể	48
2.5.7	Kết quả yêu cầu 7: Tương đồng văn bản bằng TF-IDF)	49
2.5.8	Kết quả yêu cầu 8: Phân tích chuỗi thời gian	50
2.5.9	Tóm tắt kết quả	51
3	Kết luận và Hướng phát triển	51
3.1	Thành tựu	51
3.2	Hạn chế	52
3.3	Đánh giá tổng thể	52
3.4	Định hướng phát triển	52
3.4.1	Quản lý trạng thái	52
3.4.2	Hiệu năng	53
3.4.3	Phân tích nâng cao	53
3.4.4	Hạ tầng	53

3.4.5	Giao diện người dùng	53
3.4.6	Lộ trình ưu tiên	53

4 Tài liệu Tham khảo

54

MỞ ĐẦU

Sự phát triển nhanh chóng và liên tục của các dự án phần mềm trên GitHub đặt ra nhiều cơ hội và thách thức đối với việc phân tích xu hướng phát triển công nghệ và hoạt động cộng đồng lập trình. Dự án này xây dựng một pipeline phân tích dữ liệu thời gian thực có khả năng mở rộng, sử dụng dữ liệu repositories từ GitHub API, đồng thời lưu trữ và xử lý dữ liệu lịch sử các repository thông qua môi trường dữ liệu lớn gồm Apache Spark Streaming, Redis, và Flask.

Bằng cách tận dụng dữ liệu luồng từ GitHub, hệ thống thực hiện thu thập, tiền xử lý và phân tích dữ liệu theo thời gian thực, từ đó cung cấp các thông tin giá trị về số lượng repository, số sao trung bình, từ khóa phổ biến, chủ đề nổi bật, thực thể công nghệ, độ tương đồng giữa các repository và xu hướng phát triển theo thời gian.

Các kỹ thuật chính bao gồm xây dựng pipeline dữ liệu thời gian thực để xử lý dữ liệu với tốc độ cao, quản lý trạng thái và loại bỏ trùng lặp để đảm bảo tính chính xác của phân tích, trích xuất thông tin văn bản từ mô tả repository, và áp dụng các mô hình Machine Learning như Latent Dirichlet Allocation (LDA) và TF-IDF để khám phá chủ đề, đánh giá mức độ tương đồng và xu hướng công nghệ. Việc triển khai sử dụng Spark RDD và DataFrame để xử lý phân tán, kết hợp với Redis để lưu trữ kết quả phân tích và Flask để trực quan hóa trên dashboard, đảm bảo khả năng mở rộng, độ tin cậy và khả năng khôi phục lỗi.

Kết quả của dự án chứng minh tính hiệu quả của framework được đề xuất, cung cấp thông tin phân tích repositories GitHub gần thời gian thực với độ trễ thấp. Hệ thống này không chỉ giúp theo dõi xu hướng phát triển công nghệ mà còn cung cấp cơ sở để xây dựng các ứng dụng gợi ý, dự đoán xu hướng công nghệ và nghiên cứu hành vi cộng đồng lập trình viên trên GitHub.

1 Tổng quan

1.1 Giới thiệu

1.1.1 Tại sao phân tích repositories GitHub lại quan trọng?

Phân tích repositories GitHub là một hoạt động quan trọng trong việc hiểu xu hướng phát triển công nghệ, hành vi cộng đồng lập trình viên, và mức độ phổ biến của các công

cụ, thư viện hay framework. Việc này giúp các tổ chức, nhà nghiên cứu, hoặc cá nhân đưa ra quyết định thông minh về đầu tư vào dự án phần mềm, phát triển sản phẩm mới, hoặc xây dựng chiến lược tuyển dụng, đào tạo.

Những thách thức trong môi trường dữ liệu lớn của GitHub:

- **Khối lượng dữ liệu lớn:** GitHub chứa hàng triệu repositories với dữ liệu liên tục tăng, yêu cầu hệ thống có khả năng mở rộng.
- **Độ phức tạp của dữ liệu:** Dữ liệu repository bao gồm metadata, README, issue, pull request, commit history, chứa nhiều dữ liệu bán cấu trúc hoặc không cấu trúc.
- **Tốc độ dữ liệu:** Các hoạt động trên GitHub diễn ra liên tục (real-time), đòi hỏi pipeline phân tích phải nhanh và hiệu quả.
- **Đảm bảo tính chính xác:** Dữ liệu từ nhiều repository khác nhau có thể không đồng nhất, gây khó khăn trong phân tích thống kê và học máy.
- **Quản lý dữ liệu:** Lưu trữ, bảo mật và truy xuất dữ liệu trong môi trường phân tán gặp nhiều thách thức.
- **Chi phí tính toán:** Xử lý dữ liệu lớn đòi hỏi tài nguyên tính toán và lưu trữ đáng kể.
- **Tích hợp hệ thống:** Đồng bộ hóa nhiều công cụ và nền tảng (API GitHub, Spark, Redis, Flask) đôi khi gặp vấn đề tương thích.

1.1.2 Mục tiêu cụ thể của dự án

Mục tiêu cụ thể của dự án phân tích repositories GitHub bao gồm:

- **Phát triển hệ thống phân tích thời gian thực:** Thu thập dữ liệu repositories từ GitHub API, xử lý và phân tích thông tin về số sao, số fork, contributors, chủ đề, ngôn ngữ lập trình và xu hướng phát triển.
- **Xây dựng pipeline dữ liệu hiệu quả:** Tạo hệ thống xử lý dữ liệu mạnh mẽ, có khả năng mở rộng, cập nhật liên tục và đảm bảo tính chính xác.
- **Áp dụng các kỹ thuật học máy và NLP:** Sử dụng TF-IDF và LDA để trích xuất chủ đề, phân tích similarity và khám phá xu hướng công nghệ.
- **Đảm bảo khả năng mở rộng và tính khả dụng:** Triển khai trên Spark và Redis, đảm bảo hệ thống xử lý được khối lượng dữ liệu lớn và duy trì hoạt động ổn định.

1.1.3 Phạm vi dự án

Dự án này bao gồm:

- **Thu thập và xử lý dữ liệu:** Thu thập dữ liệu repository, issue, commit từ GitHub API; làm sạch, chuẩn hóa và trích xuất đặc trưng.
- **Xây dựng pipeline dữ liệu thời gian thực:** Sử dụng Spark Streaming để xử lý dữ liệu trực tiếp, Redis để lưu trữ kết quả phân tích và Flask để hiển thị dashboard trực quan.
- **Ứng dụng mô hình học máy:** Sử dụng TF-IDF, LDA và các kỹ thuật similarity để phân tích chủ đề, mức độ phổ biến và liên kết giữa các repository.
- **Triển khai và trực quan hóa:** Tích hợp dữ liệu trên dashboard Flask, cho phép quan sát xu hướng repository, stars, forks và chủ đề nổi bật theo thời gian.

Dự án không đi sâu vào các vấn đề:

- **Phân tích dài hạn:** Không phân tích dữ liệu lịch sử quá dài, tập trung vào xu hướng hiện tại.
- **Ảnh hưởng từ yếu tố bên ngoài:** Không xem xét các yếu tố kinh tế, xã hội hoặc chính trị.
- **Đánh giá chất lượng mã nguồn:** Không kiểm tra chất lượng code chi tiết, chủ yếu dựa trên metadata và mô tả repository.
- **Triển khai đa nền tảng:** Tập trung vào GitHub, không mở rộng sang GitLab hay Bitbucket.

1.2 Một số nghiên cứu liên quan

1.2.1 Tổng quan về các nghiên cứu

Phân tích dữ liệu GitHub là chủ đề nghiên cứu ngày càng phổ biến nhờ vào khối lượng và độ đa dạng của repositories. Các nghiên cứu trước đây tập trung vào:

- **Phân tích metadata:** Dựa trên stars, forks, contributors để đánh giá mức độ phổ biến và ảnh hưởng.
- **Khám phá chủ đề:** Ứng dụng NLP trên README hoặc description để nhận diện xu hướng công nghệ.
- **Mạng lưới hợp tác:** Xây dựng graph contributors và forks để phân tích sự tương tác trong cộng đồng lập trình.

1.2.2 Các kỹ thuật phổ biến

Một số kỹ thuật phổ biến trong phân tích repository GitHub gồm:

- **TF-IDF:** Trích xuất từ khóa quan trọng từ README hoặc description.
- **LDA (Latent Dirichlet Allocation):** Phát hiện chủ đề tiềm ẩn để nhận diện xu hướng công nghệ.
- **Cosine Similarity / Jaccard Index:** Đánh giá sự tương quan giữa các repository dựa trên từ khóa, chủ đề hoặc ngôn ngữ.
- **Network Analysis:** Phân tích đồ thị contributors và forks để khám phá mối quan hệ và ảnh hưởng.

1.2.3 Các đặc trưng dùng cho phân tích

Các đặc trưng được sử dụng trong dự án bao gồm:

- **Metadata repository:** Stars, forks, watchers, số contributors, số issue mở/đóng.
- **Thông tin ngôn ngữ lập trình:** Ngôn ngữ chính và phụ.
- **Chủ đề repository:** Trích xuất từ description, README hoặc tags.
- **Thông tin commit:** Số commit, lịch sử commit, tác giả commit.

1.2.4 Vai trò của Big Data trong phân tích GitHub

Việc áp dụng Big Data giúp xử lý và phân tích dữ liệu GitHub lớn và liên tục thay đổi hiệu quả. Các công cụ như Spark Streaming và Redis cho phép:

- Thu thập dữ liệu real-time từ API GitHub.
- Xử lý dữ liệu phân tán với hiệu suất cao và khả năng mở rộng.
- Lưu trữ và truy xuất dữ liệu phân tích nhanh để hiển thị trên dashboard.

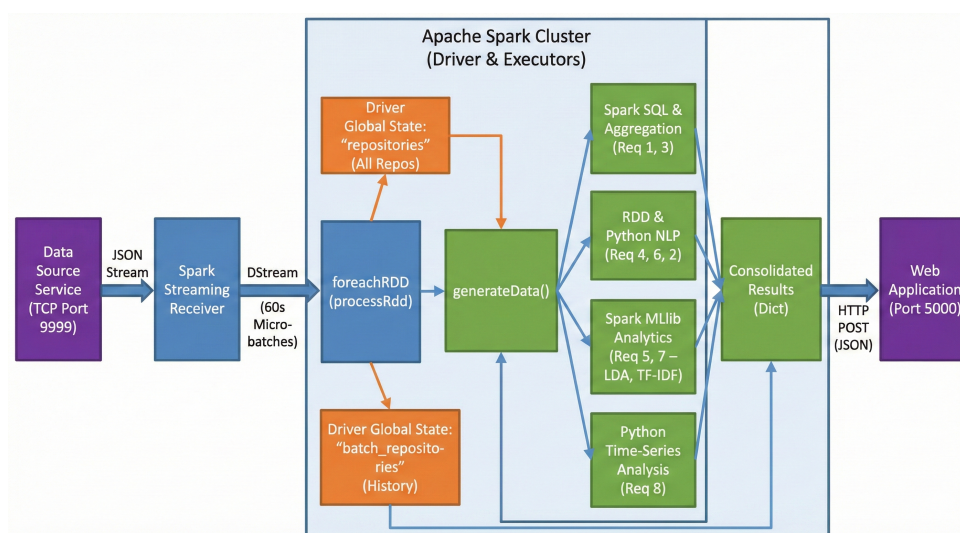
Nhờ đó, dự án cung cấp thông tin phân tích gần thời gian thực, giúp nhận diện xu hướng repository, đánh giá mức độ phổ biến và khám phá mối liên hệ giữa các dự án phần mềm.

2 Phương pháp

2.1 Các công nghệ sử dụng (Big Data)

Dự án sử dụng các công nghệ thuộc hệ sinh thái xử lý dữ liệu lớn nhằm xây dựng pipeline thu thập – xử lý – phân tích dữ liệu gần thời gian thực. Các công nghệ này được lựa chọn dựa trên khả năng phân tán, hỗ trợ xử lý streaming, khả năng mở rộng linh hoạt và dễ dàng triển khai trong môi trường container. Dưới đây là mô tả chi tiết từng công nghệ, cùng vai trò của chúng trong hệ thống.

2.1.1 Apache Spark Streaming



Giới thiệu công nghệ

Apache Spark là nền tảng xử lý dữ liệu phân tán nổi tiếng, cho phép xử lý dữ liệu ở tốc độ cao bằng cách tận dụng bộ nhớ RAM thay vì đọc/ghi liên tục xuống ổ đĩa như Hadoop MapReduce. Spark hỗ trợ nhiều mô hình xử lý như batch, interactive, machine learning và đặc biệt là Spark Streaming, mô hình xử lý dữ liệu luồng theo micro-batch.

Spark Streaming thực hiện xử lý luồng bằng cách chia dữ liệu thành các batch nhỏ (ví dụ: 1 giây hoặc 60 giây). Cách tiếp cận micro-batch này giúp hệ thống vẫn đạt gần real-time nhưng đảm bảo dễ mở rộng, độ ổn định cao, và tối ưu tài nguyên hơn so với xử lý từng sự kiện (event-by-event).

Bên cạnh đó, Spark hỗ trợ rất tốt các thư viện phân tích nâng cao như MLlib (machine learning), GraphX (phân tích đồ thị), và hệ sinh thái PySpark cho Python.

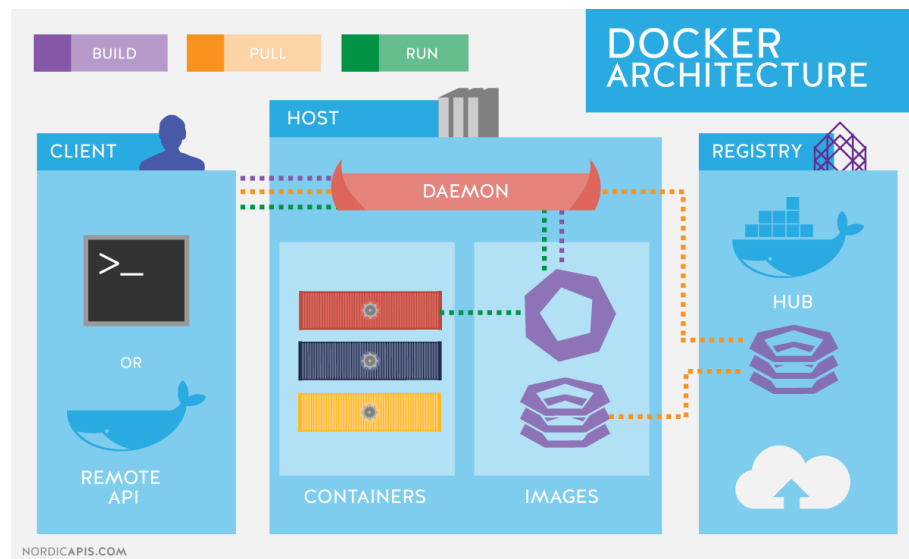
Vai trò trong dự án

Trong hệ thống này, Spark Streaming giữ vai trò là bộ xử lý trung tâm, thực hiện:

- Tiếp nhận dữ liệu repository từ GitHub API theo chu kỳ.
- Làm sạch, chuẩn hóa và xử lý văn bản mô tả repository.
- Trích xuất đặc trưng (features) như số sao, số forks, thời gian cập nhật.
- Áp dụng các kỹ thuật phân tích: TF-IDF, Cosine Similarity, LDA Topic Modeling-, NER
- Xuất kết quả phân tích sang Redis để dashboard web cập nhật.
- Lưu checkpoint để phục hồi khi container bị restart.

Spark được triển khai theo mô hình Master – Worker bằng Docker, cho phép dễ dàng mở rộng số Worker khi cần tăng tốc độ xử lý.

2.1.2 Docker & Docker Compose



Giới thiệu công nghệ

Docker là nền tảng container hóa cho phép đóng gói ứng dụng cùng toàn bộ môi trường chạy của nó (thư viện, biến môi trường, dependency). Một container hoạt động độc lập, đồng nhất trên mọi hệ thống, giúp loại bỏ tình trạng “chạy trên máy tôi được, máy bạn lỗi”.

Docker Compose là công cụ hỗ trợ chạy nhiều container cùng lúc thông qua một file cấu hình duy nhất. Nó giúp thiết lập mạng nội bộ giữa các container, kiểm soát thứ tự khởi động, scale dịch vụ và theo dõi trạng thái hệ thống.

Trong các dự án Big Data, Docker giúp mô phỏng môi trường phân tán một cách dễ dàng, không cần cluster vật lý.

Vai trò trong dự án

Docker được sử dụng để triển khai toàn bộ hệ thống, bao gồm:

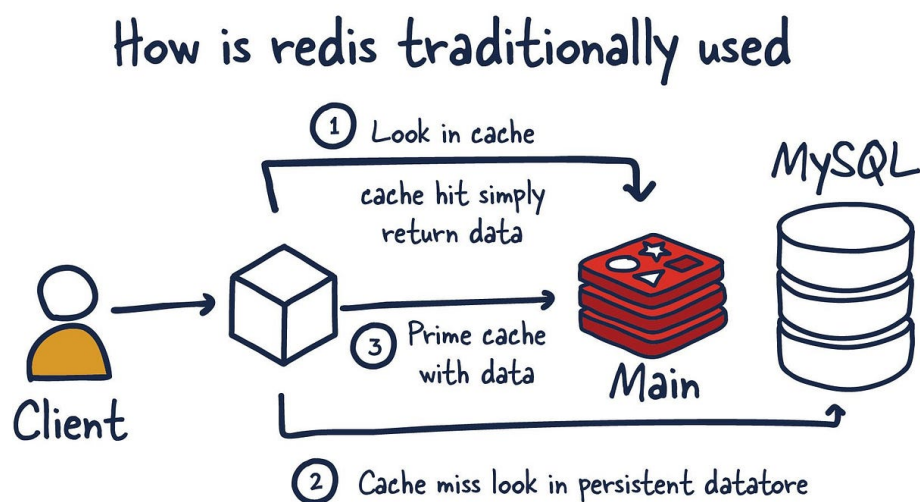
- Spark Master
- Nhiều Spark Worker
- Redis
- Webapp

Docker Compose giúp:

- Điều phối các container và đảm bảo đúng thứ tự khởi động.
- Tạo mạng nội bộ để Spark có thể giao tiếp với Worker và Redis.
- Dễ dàng scale số worker Spark khi khối lượng dữ liệu tăng.
- Giúp hệ thống thống nhất khi triển khai trên máy bất kỳ.

Nhờ Docker, pipeline Big Data có thể khởi động nhanh chỉ với một lệnh duy nhất.

2.1.3 Redis



Giới thiệu công nghệ

Redis (Remote Dictionary Server) là cơ sở dữ liệu in-memory tốc độ cao, được tối ưu cho truy vấn đọc-ghi trong vài micro-millisecond. Redis lưu dữ liệu trong RAM thay vì lưu trên ổ đĩa, nhờ vậy rất phù hợp cho các ứng dụng real-time như caching, pub/sub, hàng đợi tác vụ và lưu trữ tạm thời.

Redis hỗ trợ nhiều cấu trúc dữ liệu như string, list, hash, sorted set, JSON, giúp nó linh hoạt trong việc lưu trữ các dạng dữ liệu khác nhau.

Vai trò trong dự án

Trong hệ thống này, Redis được sử dụng làm lớp lưu trữ tạm:

- Nhận dữ liệu phân tích gửi từ Spark.
- Làm cache để trả kết quả cho webapp nhanh hơn.
- Lưu các số liệu thống kê gần nhất để dashboard hiển thị real-time.
- Giúp giảm tải cho Spark khi dashboard truy vấn liên tục.

Redis đảm bảo toàn bộ dashboard hoạt động mượt mà, luôn hiển thị dữ liệu mới nhất mà không gây ảnh hưởng tới pipeline xử lý.

2.1.4 GitHub API (Nguồn dữ liệu streaming)

Giới thiệu công nghệ

GitHub API là giao diện lập trình cho phép truy cập hàng triệu repository mã nguồn mở. API hỗ trợ truy vấn dữ liệu có cấu trúc như tên dự án, số sao, mô tả, ngôn ngữ lập trình, thời gian cập nhật, chủ sở hữu, watchers,...

GitHub API hỗ trợ phân trang, lọc theo ngôn ngữ, sort theo thời gian cập nhật, nên rất phù hợp cho các hệ thống cần dữ liệu biến động liên tục. Khi sử dụng kèm GitHub Personal Access Token (PAT), hệ thống tăng được giới hạn rate-limit truy vấn.

Vai trò trong dự án

GitHub API đóng vai trò nguồn dữ liệu đầu vào, cung cấp dòng dữ liệu cập nhật theo thời gian:

- Thu thập repository mới cập nhật trong các ngôn ngữ Python, Java, JavaScript.
- Gửi dữ liệu thô sang Spark Streaming để phân tích.
- Là nguồn dữ liệu chính của pipeline near real-time.

API đảm bảo tính liên tục của dòng dữ liệu, giúp mô hình luôn xử lý thông tin mới.

2.2 Thu thập và Xử lý Dữ liệu

Hệ thống thu thập dữ liệu các kho lưu trữ GitHub và xử lý theo thời gian thực bằng Apache Spark. Dữ liệu đi từ GitHub API qua socket TCP đến Spark, nơi dữ liệu được xử lý theo mô hình micro-batch.

2.2.1 Thu thập Dữ liệu

Kiến trúc Dịch vụ Thu thập Dữ liệu

Dịch vụ thu thập dữ liệu (`data_source.py`) chạy bên trong container Docker và liên tục lấy dữ liệu repository từ GitHub API.

Công nghệ sử dụng:

- Python 3.9
- requests (HTTP)
- socket (TCP streaming)
- GitHub REST API

Quy trình Thu thập Dữ liệu

Chiến lược Gọi API

- Ngôn ngữ: Python, Java, JavaScript
- Gọi API tuần tự theo từng ngôn ngữ
- Endpoint: `https://api.github.com/search/repositories?q=language:{language}&sort=`
- Chu kỳ truy vấn: 15 giây
- Tối đa 50 repository mỗi ngôn ngữ

Xác thực

- Sử dụng GitHub Personal Access Token (PAT)
- Token lấy từ biến môi trường `TOKEN`
- Header: `"token "+ os.getenv('TOKEN')`
- Token được khai báo trong `docker-compose.yaml` (giá trị dummy khi nộp bài)

Lọc và Kiểm tra Dữ liệu

```
1 if repository["language"] is None or repository["language"] not in
   LANGUAGES:
2     continue
```

- Loại bỏ repository không thuộc 3 ngôn ngữ mục tiêu
- Bỏ qua repository có ngôn ngữ None
- Đảm bảo chất lượng dữ liệu trước khi truyền đi

Cấu trúc Dữ liệu Trích Xuất

Mỗi repository bao gồm:

- **id**: Mã định danh (dùng để deduplication)
- **name**: Tên repository
- **language**: Ngôn ngữ chính
- **description**: Mô tả repository
- **stargazers_count**: Số lượng stars
- **pushed_at**: Thời điểm cập nhật cuối

Cơ chế Streaming Dữ liệu

TCP Socket Server

- Liên kết tại 0.0.0.0:9999 (lắng nghe trên tất cả các giao diện)
- Sử dụng `socket.accept()` chặn để chờ kết nối Spark
- Mô hình kết nối: 1 kết nối duy nhất với Spark
- Định dạng: NDJSON (newline-delimited JSON)

Định dạng Truyền Dữ liệu

```
1 json_data = f"{json.dumps(data)}\n".encode()
2 conn.send(json_data)
```

- Mỗi repository được gửi dưới dạng một đối tượng JSON độc lập.
- Ký tự xuống dòng (`\n`) được dùng để phân tách các bản ghi.
- Dữ liệu được mã hóa nhị phân để truyền qua mạng.
- Dữ liệu được truyền liên tục (không gom batch ở phía nguồn).

Chu kỳ Thu thập

- Thiết lập máy chủ TCP và chờ kết nối từ Spark.
- Vào vòng lặp vô hạn:
 - Với mỗi ngôn ngữ trong ['Python', 'Java', 'JavaScript']:
 - * Gửi truy vấn GitHub API với bộ lọc ngôn ngữ.
 - * Phân tích phản hồi JSON.
 - * Lọc repository theo ngôn ngữ.
 - * Gửi từng repository dưới dạng JSON line qua TCP.
 - Nghỉ 15 giây.
 - Lặp lại.

Đặc tính Throughput

- Thông lượng tiềm năng: khoảng 150 repository trong mỗi chu kỳ 15 giây (50 mỗi ngôn ngữ $\times$ 3 ngôn ngữ).
- Thông lượng thực tế: phụ thuộc mức độ hoạt động trên GitHub tại thời điểm truy vấn.
- Hiệu quả mạng: sử dụng một kết nối TCP duy nhất, giảm tối thiểu overhead truyền tải.

2.2.2 Xử lý Dữ liệu

Kiến trúc Spark Streaming

Cấu hình Streaming Context

```
1 BATCH_INTERVAL = 60 # 60-second micro-batches
2 ssc = StreamingContext(sc, BATCH_INTERVAL)
3 ssc.checkpoint("checkpoint_EECS4415_Project_3")
```

- Chu kỳ xử lý (batch interval): 60 giây
- Mô hình xử lý: Micro-batch streaming (DStreams)
- Checkpointing: Giúp phục hồi trạng thái khi xảy ra lỗi
- Cấp lưu trữ: MEMORY_AND_DISK (cân bằng hiệu năng và độ tin cậy)

Thu thập Dữ liệu (Data Ingestion)

```
1 data = ssc.socketTextStream(  
2     DATA_SOURCE_IP,  
3     DATA_SOURCE_PORT,  
4     storageLevel=StorageLevel.MEMORY_AND_DISK  
5 )
```

- Kết nối: Nhận dữ liệu qua TCP socket từ `data-source:9999`
- Định dạng đầu vào: Dữ liệu dạng văn bản (JSON từng dòng)
- Lưu trữ: Tự động ghi xuống disk nếu RAM không đủ
- Kiểu luồng: Liên tục, không giới hạn thời gian

Pipeline Biến đổi Dữ liệu (Data Transformation Pipeline)

Xử lý JSON

```
1 repos = data.flatMap(lambda json_str: [json.loads(json_str)])
```

- **flatMap**: Chuyển từng dòng JSON thành dictionary Python
- Xử lý lỗi: Trả về danh sách rỗng nếu dữ liệu lỗi (tự loại bỏ)
- Đầu ra: DStream chứa các dictionary repository

Kích hoạt Xử lý Batch

```
1 repos.foreachRDD(processRdd)
```

- **foreachRDD**: Phép toán đầu ra kích hoạt xử lý cho từng batch
- Chu kỳ chạy: 60 giây một lần
- Đầu vào: Một RDD chứa toàn bộ dữ liệu nhận được trong 60 giây
- Quy mô dữ liệu mong đợi: 0–600 repository mỗi chu kỳ (tùy hoạt động của GitHub)

Quản lý Trạng thái và Loại bỏ Trùng Lặp

Biến Trạng Thái Toàn Cục

- **repositories**: Lưu toàn bộ repository duy nhất theo ID (toàn hệ thống)
- **batch_repositories**: Lưu repository của từng batch kèm timestamp
- **spark_session**: SparkSession dùng cho DataFrame
- **SparkContext**: Ngữ cảnh xử lý RDD
- **startTime**: Thời điểm bắt đầu phiên xử lý

Chiến lược Loại Bỏ Trùng Lặp

```
1 for repo in rdd.collect():
2     if repo['id'] not in batch_repositories:
3         batch_repositories[repo['id']] = repo
4     if repo['id'] not in repositories:
5         repositories[repo['id']] = repo
```

- Khóa định danh: **ID repository** (duy nhất)
- Cấp batch: Ngăn trùng lặp trong phạm vi mỗi batch 60 giây
- Cấp toàn hệ thống: Đảm bảo mỗi repository chỉ được tính một lần duy nhất
- Cách thực hiện: Lookup dictionary với độ phức tạp $O(1)$
- Áp dụng: Ngay tại bước ingestion trước khi phân tích

Lưu Trữ Trạng Thái

- Dictionary toàn cục giữ nguyên giữa các batch
- Checkpointing cho phép phục hồi khi driver gặp lỗi
- Nhược điểm: Dữ liệu trong RAM tăng dần theo thời gian – phù hợp cho luồng dài nhưng không vô hạn

2.2.3 Quy trình Xử lý

Hàm Xử lý Batch (processRdd)

Hàm `processRdd(time, rdd)` điều phối xử lý cho mỗi batch:

1. Thu thập và Khử trùng lặp Repository

- Chuyển RDD sang danh sách Python (đủ nhỏ để lưu trong bộ nhớ)
- Khử trùng lặp ở cấp batch và cấp toàn hệ thống
- Cập nhật bộ nhớ cache repository toàn cục

2. Cập nhật Trạng thái

- `updateAllRepositories()`: cập nhật dictionary repository toàn cục
- `updateBatchRepositories()`: thêm batch vào lịch sử kèm timestamp UTC cho phân tích chuỗi thời gian

3. Thực thi Phân tích

- Gọi `generateData()` kích hoạt 8 hàm phân tích
- Kết quả được gom thành một dictionary duy nhất
- Gửi về ứng dụng web qua HTTP POST

Xử lý Lỗi

```
1 try:
2     # Processing logic
3 except Exception as e:
4     print("An error occurred:", e)
5     traceback.print_exc()
6     print("Waiting for Data...")
```

- Câu lệnh try-except giúp bắt lỗi trong quá trình xử lý
- Hệ thống tiếp tục chạy ngay cả khi một batch thất bại
- Ghi log traceback để phục vụ debug
- Suy giảm chất lượng hoạt động một cách an toàn nếu các hàm phân tích lỗi

Kỹ thuật Xử lý Dữ liệu

Kết hợp API

DataFrames: dùng cho các phép tổng hợp dạng SQL (yêu cầu 3.1, 3.3 trong Readme.md)

- Ví dụ:

```
1 df.groupBy('language').count()
```

- Lợi ích:

- Tối ưu hóa execution plan
- Cú pháp giống SQL

- **RDDs:** dùng cho xử lý dữ liệu chi tiết (yêu cầu 3.2, 3.4 trong Readme.md)

```
1 repos.map(lambda repo: (repo['language'], 1)).reduceByKey(lambda a, b: a+b)
```

- Lợi ích:

- Kiểm soát chi tiết
- Tùy biến cao

Pipeline Xử lý Văn bản

Dùng cho phân tích văn bản (yêu cầu Top 10 từ phổ biến):

```
1 words_by_language = repos.flatMap(  
2     lambda repo: ((repo['language'], word)  
3     for word in re.sub('[^a-zA-Z ]', '', str(repo['description']))  
4     .lower().split()  
5     if repo['description'] is not None)  
6 ).groupByKey()
```

- Tiền xử lý văn bản: loại ký tự không phải chữ, chuyển lowercase, token hóa
- Xử lý phân tán: sử dụng thao tác RDD để tăng khả năng mở rộng
- Đếm từ: dùng Python Counter để thống kê tần suất
- Lọc Top-K: sắp theo tần suất và chọn 10 từ cao nhất

Lọc theo Thời gian

Dùng cho yêu cầu 3.2 (Các repository được push gần đây):

```
1 pushed_at = datetime.strptime(repo['pushed_at'], '%Y-%m-%dT%H:%M:%S')
2 delta = batch_time - pushed_at
3 if delta.total_seconds() <= 60:
4     current_batch_repos_last_60[repo['id']] = repo
```

- Phân tích timestamp chuẩn ISO 8601
- Tính độ lệch thời gian giữa batch và thời điểm push repository
- Lọc repository được push trong vòng 60 giây
- Đảm bảo tính thời gian thực cho phân tích streaming

Xuất Dữ Liệu và Phân Phối

Tổng hợp Kết quả

```
1 data = {
2     'req1': getTotalRepoCountByLanguage(),
3     'req2': getBatchedRepoLanguageCountsLast60Seconds(),
4     'req3': getAvgStargazersByLanguage(),
5     'req4': getTopTenFrequentWordsByLanguage(),
6     'req5': getTopicModelingByLanguage(),
7     'req6': getNamedEntityRecognition(),
8     'req7': getTFIDFCosineSimilarity(),
9     'req8': getTimeSeriesAnalysis()
10 }
```

- Tất cả 8 kết quả phân tích được tổng hợp vào một dictionary duy nhất.
- Định dạng có cấu trúc, dễ dàng tiêu thụ bởi WebApp.

Truyền Dữ liệu sang Webapp

```
1 def sendToClient(data):
2     url = 'http://webapp:5000/updateData'
3     response = requests.post(url, json=data)
```

Thông số truyền tải:

- Giao thức: HTTP REST API
- Phương thức: POST
- Endpoint: /updateData
- Định dạng dữ liệu: JSON payload
- Tần suất: Sau mỗi batch 60 giây
- Xử lý lỗi: Bắt ngoại lệ, ghi log và tiếp tục streaming

Xuất Console

- Mỗi batch in ra các kết quả theo yêu cầu.
- Hiển thị Spark DataFrame bằng `.show()`.
- In thời gian chạy, số lượng RDD và timestamp của batch.
- Dễ kiểm chứng bằng mắt người dùng.

2.2.4 Tổng Quan Luồng Dữ Liệu

Luồng Tổng Thể

GitHub API

↓ (mỗi 15 giây, 3 ngôn ngữ × 50 repo)

Data Source Service (data_source.py)

↓ (TCP Socket, Port 9999, NDJSON format)

Spark Streaming (spark_app.py)

↓ (micro-batches 60 giây)

Khử trùng lặp (theo Repository ID)

Quản lý trạng thái (Dictionary toàn cục và theo batch)

Thực thi phân tích (8 tác vụ)

Tổng hợp kết quả

↓ (HTTP POST, JSON Payload)

Webapp Service

↓ (Redis caching)

Frontend Dashboard

Đặc điểm Thời gian

Giai đoạn	Chu kỳ	Thông lượng
GitHub API Queries	15 giây	~150 repo/chu kỳ
Spark Batch Processing	60 giây	0–600 repo/batch
Result Transmission	60 giây	1 bộ kết quả/batch
Frontend Refresh	60 giây	1 lần cập nhật visualization

Ước Tính Dữ liệu

- Mỗi chu kỳ 15 giây: ≈ 150 repository (50 mỗi ngôn ngữ $\times 3$).
- Mỗi batch 60 giây: ≈ 600 repository.
- Mỗi giờ: $\approx 36,000$ repository (60 batch $\times 600$ repo).
- Dữ liệu tăng tuyến tính theo số lượng repository mới (nhờ deduplication).

2.2.5 Các Quyết Định Thiết Kế

Chiến lược Thu thập

- **Quyết định:** Gọi API tuần tự theo ngôn ngữ.
- **Lý do:** Phù hợp yêu cầu dự án và logic phân tách theo ngôn ngữ.
- **Đánh đổi:** Chậm hơn xử lý song song nhưng đơn giản và ổn định hơn.

Giao thức Streaming

- **Quyết định:** TCP + NDJSON.
- **Lý do:** Nhẹ, hỗ trợ tốt trong Spark.
- **Phương án thay thế:** Kafka/RabbitMQ (mạnh hơn nhưng phức tạp hơn).

Chu kỳ Batch

- **Quyết định:** Batch 60 giây.
- **Lý do:** Cân bằng giữa độ trễ và hiệu quả xử lý.
- **Đánh đổi:** Chưa đạt real-time hoàn toàn (sub-second), nhưng đủ cho phân tích nặng.

Quản lý Trạng thái

- **Quyết định:** Dictionary Python giữ trạng thái toàn cục.
- **Lý do:** Đơn giản và hiệu quả cho ứng dụng streaming một máy.
- **Đánh đổi:** Trạng thái mất nếu driver chết (giảm thiểu nhờ checkpointing).

Chiến lược Khử trùng lặp

- **Quyết định:** Khử trùng lặp theo repository ID bằng dictionary.
- **Lý do:** Lookup $O(1)$, dễ triển khai, thực thi ngay tại ingestion.
- **Đánh đổi:** Trạng thái tăng dần theo thời gian nhưng chậm và có thể kiểm soát.

2.2.6 Độ Tin Cậy và Khả năng Chịu lỗi

Checkpointing

- Spark Streaming checkpoint lưu lineage và trạng thái của DStream.
- Giúp khôi phục nếu driver gặp sự cố.

Mức Lưu Trữ

- **MEMORY_AND_DISK**: đảm bảo dữ liệu không bị mất.
- Tự động ghi ra ổ cứng nếu thiếu RAM.
- Cân bằng giữa tốc độ và độ tin cậy.

Xử lý Lỗi

- Khởi **try-except** trong `processRdd()` để bắt lỗi.
- Streaming tiếp tục ngay cả khi batch bị lỗi.
- Ghi lại `traceback` để debug.
- Ứng dụng xuống cấp duyên dáng (*graceful degradation*).

Khả năng chịu mất kết nối

- Data source chờ Spark kết nối bằng `accept()` blocking.
- Nếu Spark mất kết nối, tiến trình chờ kết nối lại.
- Checkpoint của Spark đảm bảo phục hồi sau lỗi.

2.2.7 Kết luận

Phần Thu thập và Xử lý Dữ liệu xây dựng pipeline phân tích GitHub thời gian thực. Dữ liệu được thu thập mỗi 15 giây, truyền qua TCP, xử lý bởi Spark mỗi 60 giây, được phân tích và gửi tới webapp để hiển thị. Hệ thống cân bằng giữa thời gian thực và độ phức tạp phân tích, với checkpointing và xử lý lỗi giúp tăng độ tin cậy.

2.3 Phương pháp tổng hợp và phân tích

Hệ thống thực hiện 8 tác vụ phân tích trên dữ liệu lưu trữ GitHub đang phát trực tuyến. Mỗi tác vụ sử dụng Spark (`DataFrame` hoặc `RDD`) và chạy mỗi 60 giây. Phương pháp này sử dụng công nghệ loại bỏ trùng lặp, quản lý trạng thái và xử lý phân tán để tạo ra thông tin chi tiết chính xác, theo thời gian thực.

2.3.1 Các Nhiệm Vụ Phân Tích Chính

Yêu cầu 1: Tổng số kho theo ngôn ngữ

Mục tiêu: Đếm tổng số kho (repository) duy nhất cho từng ngôn ngữ lập trình trong toàn bộ vòng đời luồng dữ liệu.

Triển khai:

```
1 def getTotalRepoCountByLanguage():
2     repos_list = list(getAllRepositories().values())
3     df = spark.createDataFrame(repos_list)
4     counts_df = df.groupBy('language').count()
```

Các bước xử lý:

1. **Nguồn dữ liệu:** Lấy toàn bộ kho đã thu thập từ trạng thái toàn cục.
2. **Tạo DataFrame:** Chuyển danh sách repository Python thành Spark DataFrame.
3. **Gộp nhóm:** Nhóm theo cột *language* và đếm số lượng.
4. **Định dạng đầu ra:** Danh sách các dictionary: {'language': str, 'count': int}.

Các đặc điểm chính:

- **Loại bỏ trùng lặp:** Mỗi repository chỉ được tính đúng một lần.
- **Spark API:** Sử dụng các thao tác DataFrame tối ưu.
- **Độ bao phủ ngôn ngữ:** Đảm bảo cả ba ngôn ngữ xuất hiện trong kết quả.
- **Ví dụ đầu ra:** [{'language': 'Python', 'count': 7369}, ...].

Hiệu năng:

- **Hiệu quả:** GroupBy chạy trực tiếp trên DataFrame.
- **Mở rộng tốt:** Có thể chạy phân tán trên cụm Spark.
- **Bộ nhớ:** Cần thu thập toàn bộ kho về driver (chấp nhận được với dữ liệu vừa phải).

Yêu cầu 2: Các lần cập nhật gần nhất (60 giây gần nhất)

Mục tiêu: Đếm số repository được cập nhật trong 60s gần nhất, nhóm theo ngôn ngữ cho mỗi batch xử lý.

Triển khai:

```

1 def getBatchedRepoLanguageCountsLast60Seconds():
2     current_batch = getBatchRepositories()[-1]
3     batch_time = current_batch['time']
4
5     for repo in current_batch['repos'].values():
6         pushed_at = datetime.strptime(repo['pushed_at'], '%Y-%m-%dT%H:%M:%SZ')
7         delta = batch_time - pushed_at
8         if delta.total_seconds() <= 60:
9             current_batch_repos_last_60[repo['id']] = repo
10
11 repos = sc.parallelize(current_batch_repos_last_60.values())
12 counts = repos.map(lambda repo: (repo['language'], 1)).
    reduceByKey(lambda a, b: a+b)

```

Các bước xử lý:

1. **Đọc batch:** Lấy batch mới nhất kèm timestamp UTC.
2. **Tính thời gian chênh lệch:** So sánh thời gian xử lý và thời gian pushed_at.
3. **Lọc cửa sổ 60 giây:** Giữ các repository có delta <= 60s.
4. **Loại bỏ trùng trong batch:** Đảm bảo mỗi kho chỉ xuất hiện tối đa một lần.
5. **RDD:** Chuyển sang RDD rồi map-reduce theo ngôn ngữ.
6. **Xử lý thiếu ngôn ngữ:** Những ngôn ngữ không xuất hiện được gán 0.
7. **Lưu lịch sử:** Ghi vào danh sách BatchedRepoLanguageCounts.

Đặc điểm:

- **Độ chính xác thời gian:** Sử dụng timestamp batch cho cửa sổ 60s.
- **Loại trùng lặp ở batch:** Đảm bảo mỗi ID được tính một lần.
- **RDD:** Phù hợp với xử lý phân tán.

Định dạng đầu ra:

```
[{'batch_time': 'HH:MM:SS', 'language': str, 'count': int}, ...]
```

Xem xét hiệu năng:

- **Hiệu quả:** Chỉ xử lý dữ liệu của batch hiện tại.
- **Phức tạp O(n)** với n là số repository của batch.
- **Lịch sử tăng tuyến tính** — chấp nhận được cho trực quan hoá.

Yêu cầu 3: Số lượng sao trung bình theo ngôn ngữ

Mục tiêu: Tính số sao trung bình trên mỗi repository cho từng ngôn ngữ lập trình.

Triển khai:

```
1 def getAvgStargazersByLanguage():
2     repos_list = list(getAllRepositories().values())
3     df = spark.createDataFrame(repos_list)
4     avg_df = df.groupBy('language').agg(
5         round(avg("stargazers_count"), 2)
6         .alias('avg_stargazers_count')
7     )
```

Các bước xử lý:

1. Lấy toàn bộ repository trong trạng thái toàn cục.
2. Chuyển về dạng Spark DataFrame.
3. GroupBy theo ngôn ngữ và lấy trung bình `stargazers\_count`.
4. Làm tròn đến 2 chữ số thập phân.
5. Xuất dưới dạng

```
{'language': str, 'avg_stargazers_count': float}
```

Đặc điểm:

- Tính trên toàn bộ kho — không chỉ batch hiện tại.
- Không trùng lặp — mỗi repository đóng góp một lần.
- Tận dụng tối ưu hóa của Spark DataFrame.
- Hiển thị dễ đọc với 2 chữ số thập phân.

Công thức toán học:

$$avg_stargazers_count[language] = \frac{\sum(stargazers_count)}{count(repositories)}$$

Hiệu năng:

- GroupBy một lần nên rất hiệu quả.
- Có thể mở rộng trên cụm phân tán.
- Cần thu thập toàn bộ kho về driver (chấp nhận được).

Yêu cầu 4: Top 10 từ xuất hiện nhiều nhất theo ngôn ngữ

Mục tiêu: Trích xuất và xếp hạng 10 từ xuất hiện nhiều nhất trong phần mô tả của các repository theo từng ngôn ngữ lập trình.

Triển khai

```
1 def getTopTenFrequentWordsByLanguage():
2     repos = sc.parallelize(repositories.values())
3
4     words_by_language = repos.flatMap(
5         lambda repo: ((repo['language'], word)
6             for word in re.sub('[^a-zA-Z ]', '', str(repo['
7                 description'])).
8                 .lower().split()
9             ) if repo['description'] is not None else []
10    ).groupByKey()
11
12    word_count_by_language = words_by_language \
13        .mapValues(lambda words: Counter(words)) \
14        .mapValues(lambda word_count: sorted(
15            word_count.items(),
16            key=lambda x: x[1],
17            reverse=True
18        )[:10])
```

Quy trình xử lý

1. **Tạo RDD:** Chuyển danh sách repository vào RDD bằng `sc.parallelize()`.
2. **Trích xuất văn bản:** Sử dụng `flatMap()` để tách mô tả thành các cặp:
 $(language, word)$
3. **Tiền xử lý văn bản:**
 - Loại ký tự không phải chữ cái: `re.sub('[^a-zA-Z ]', '', ...)`
 - Chuyển thường: `.lower()`
 - Tách từ: `.split()`
 - Bỏ các mô tả rỗng hoặc None
4. **Nhóm theo ngôn ngữ:** Sử dụng `groupByKey()`.
5. **Đếm tần suất:** `mapValues(Counter)` tạo bảng đếm từ theo từng ngôn ngữ.
6. **Lựa chọn Top-10:** Sắp xếp giảm dần theo số lần xuất hiện và lấy 10 từ đứng đầu.

7. Định dạng đầu ra:

```
[
  {
    'language': 'Python',
    'top_ten_words': [('for', 1058), ('a', 889), ...]
  },
  ...
]
```

Đặc điểm nổi bật

- **Xử lý phân tán:** Các thao tác thực hiện trên RDD giúp tăng khả năng mở rộng với dữ liệu lớn.
- **Tiền xử lý chuẩn hóa:**
 - Loại dấu câu và ký tự lạ
 - Chuyển chữ thường
 - Tách từ
- **Phân tích tần suất:** Sử dụng `Counter` của Python để đếm nhanh và tối ưu.
- **Thuật toán Top-K:** Sắp xếp và chọn 10 từ có tần suất cao nhất cho mỗi ngôn ngữ.
- **Tính tổng hợp:** Xử lý trên toàn bộ mô tả (không lấy mẫu).

Pipeline xử lý văn bản

Description

- Remove Non-Alphabetic
- Lowercase
- Tokenize
- Filter None
- (language, word) pairs
- groupByKey
- Counter
- Sort
- Top 10

Hiệu năng

- **Khả năng mở rộng:** Toàn bộ pipeline chạy phân tán trên cụm Spark.

- **Tiết kiệm bộ nhớ:** Dữ liệu được xử lý dạng streaming, không cần load toàn bộ vào RAM.

- **Độ phức tạp xử lý văn bản:**

$$O(n \times m)$$

với:

- n : số repository
- m : số từ trong mô tả

- **Lựa chọn Top-K:**

$$O(k \log k)$$

với k là số từ độc nhất trong mỗi ngôn ngữ.

2.3.2 Các nhiệm vụ phân tích nâng cao

Yêu cầu 5: Mô hình Chủ đề (LDA)

Mục tiêu: Khám phá các chủ đề tiềm ẩn trong mô tả repository bằng phương pháp Latent Dirichlet Allocation (LDA).

Triển khai

```

1 def getTopicModelingByLanguage():
2     # Text preprocessing pipeline
3     tokenizer = Tokenizer(inputCol="description", outputCol="words")
4     remover = StopWordsRemover(inputCol="words", outputCol="filtered_words")
5     cv = CountVectorizer(inputCol="filtered_words",
6                          outputCol="raw_features",
7                          vocabSize=1000, minDF=2)
8     idf = IDF(inputCol="raw_features", outputCol="features")
9
10    # LDA model training
11    lda = LDA(k=5, maxIter=10, optimizer="online")
12    lda_model = lda.fit(rescaled_data)
13    topics = lda_model.describeTopics(5)

```

Quy trình xử lý

1. **Lọc dữ liệu:** Chỉ giữ các repository có mô tả và thuộc các ngôn ngữ mục tiêu.
2. **Xử lý theo từng ngôn ngữ:** Mỗi ngôn ngữ được xử lý độc lập.

3. Tiền xử lý văn bản:

- **Tách từ:** `Tokenizer` chia mô tả thành các token.
- **Loại bỏ stopwords:** `StopWordsRemover` lọc các từ phổ biến như “the”, “a”, “is”, ...
- **Tần suất thuật ngữ:** `CountVectorizer` xây dựng vector tần suất từ (vocab tối đa 1000).
- **Trọng số TF-IDF:** IDF gán trọng số theo tần suất ngược tài liệu.

4. Huấn luyện mô hình LDA:

- Số chủ đề: $k = 5$ trên mỗi ngôn ngữ
- Số vòng lặp: $maxIter = 10$
- Optimizer: `online` (thích hợp cho streaming)

5. Trích xuất chủ đề: Lấy 5 từ tiêu biểu cho từng chủ đề kèm trọng số.

6. Định dạng đầu ra: Dạng:

```
[
  {
    'language': 'Python',
    'topics': [
      {'topic_id': 0, 'top_words': [...], 'word_weights': [...]},
      ...
    ]
  },
  ...
]
```

Các đặc điểm nổi bật

- **Tích hợp với MLlib:** Sử dụng LDA của Spark ML.
- **TF-IDF:** Giảm ảnh hưởng của các từ quá phổ biến.
- **Mỗi ngôn ngữ một mô hình riêng:** Các mô hình LDA độc lập.
- **Yêu cầu dữ liệu tối thiểu:** Cần ít nhất 5 repository trên mỗi nhóm.
- **Giới hạn từ vựng:** Lấy 1000 từ phổ biến nhất (`vocabSize=1000`).
- **Số lần xuất hiện tối thiểu:** Từ phải xuất hiện ít nhất 2 document (`minDF=2`).

Thuật toán LDA

- **Mục tiêu:** Tìm các chủ đề ẩn trong tập văn bản.
- **Đầu ra:** Phân phối xác suất từ trên mỗi chủ đề.
- **Ý nghĩa:** Chủ đề thường đại diện cho nhóm nội dung như:
 - AI/ML
 - Mobile Development
 - Web Development

Hiệu năng

- **Độ phức tạp tính toán:**

$$O(k \times n \times m \times iterations)$$

với:

- k : số chủ đề
 - n : số tài liệu
 - m : số từ trong từ vựng
- **Bộ nhớ:** Cần lưu mô hình và vector TF-IDF.
 - **Khả năng mở rộng:** Optimizer dạng **online**, phù hợp với streaming.
 - **Yêu cầu dữ liệu:** Cần đủ số mô tả để mô hình tìm ra chủ đề.

Yêu cầu 6: Nhận diện Thực thể (Named Entity Recognition - NER)

Mục tiêu: Trích xuất và đếm số lần xuất hiện của các công ty, framework, công cụ và công nghệ từ phần mô tả của từng kho mã.

Triển khai:

```
1 def getNamedEntityRecognition():
2     tech_patterns = {
3         'companies': ['google', 'microsoft', 'amazon', ...],
4         'frameworks': ['react', 'angular', 'django', ...],
5         'tools': ['docker', 'kubernetes', 'redis', ...],
6         'technologies': ['ai', 'ml', 'blockchain', ...]
7     }
8
9     for repo in repos_lang:
10         description = repo['description'].lower()
11         for entity_type, patterns in tech_patterns.items():
12             for pattern in patterns:
```

```

13         if pattern in description:
14             language_entities['entities'][entity_type][
                pattern] += 1

```

Các bước xử lý:

1. **Từ điển thực thể:** Định nghĩa trước các mẫu từ khoá thuộc 4 loại:
 - Công ty: Google, Microsoft, Amazon, Facebook, Apple, v.v.
 - Framework: React, Angular, Django, Spring, TensorFlow, v.v.
 - Công cụ: Docker, Kubernetes, Redis, PostgreSQL, AWS, v.v.
 - Công nghệ: AI/ML, blockchain, IoT, microservices, v.v.
2. **So khớp mẫu:** So khớp chuỗi không phân biệt chữ hoa - thường.
3. **Đếm tần suất:** Số lần xuất hiện của mỗi thực thể trong mô tả kho mã.
4. **Chọn Top-K:** Trả về tối đa 5 thực thể phổ biến nhất trên mỗi loại.
5. **Định dạng kết quả:**

```

[{'language': str,
  'entities': {
    'companies': [...],
    'frameworks': [...],
    ...
  }
}]

```

Các đặc điểm chính:

- **Dựa trên từ điển:** Không dùng mô hình học máy, chỉ so khớp từ khóa.
- **Không phân biệt hoa thường.**
- **Phân loại theo nhóm:** 4 nhóm thực thể.
- **Xếp hạng tần suất:** Chỉ trả về các thực thể xuất hiện nhiều nhất.
- **Khám phá hệ sinh thái công nghệ:** Hiển thị stack phổ biến theo từng ngôn ngữ.

Đánh giá hiệu năng:

- **Độ phức tạp:** $O(n \times m \times p)$
Trong đó: n = số repo, m = số loại thực thể, p = số mẫu mỗi loại.
- **Dung lượng bộ nhớ:** Thấp.
- **Khả năng mở rộng:** Tuyến tính theo số kho mã.
- **Độ chính xác:** Phụ thuộc vào độ bao phủ của từ điển (có thể bỏ sót từ mới).

Yêu cầu 7: Tương đồng TF-IDF và Cosine Similarity

Mục tiêu: Xác định các kho mã tương tự nhau dựa trên nội dung mô tả bằng cách vector hóa TF-IDF và tính độ tương đồng cosine.

Triển khai:

```
1 def getTFIDFCosineSimilarity():
2     # TF-IDF pipeline
3     tokenizer = Tokenizer(inputCol="description", outputCol="words")
4     remover = StopWordsRemover(inputCol="words", outputCol="filtered_words")
5     cv = CountVectorizer(inputCol="filtered_words",
6                          outputCol="raw_features",
7                          vocabSize=1000, minDF=1)
8     idf = IDF(inputCol="raw_features", outputCol="features")
9
10    # Cosine similarity calculation
11    dot_product = sum(a * b for a, b in zip(vec1_list, vec2_list))
12    magnitude1 = sum(a * a for a in vec1_list) ** 0.5
13    magnitude2 = sum(a * a for a in vec2_list) ** 0.5
14    cosine_sim = dot_product / (magnitude1 * magnitude2)
```

Quy trình xử lý:

1. **Lọc dữ liệu:** Chỉ giữ các repo có mô tả và thuộc ngôn ngữ mục tiêu.
2. **Xử lý theo từng ngôn ngữ độc lập.**
3. **Vector hóa TF-IDF:**
 - Tách từ (*Tokenizer*).
 - Loại bỏ stop-word.
 - Tạo TF vector bằng *CountVectorizer*.
 - Chuyển thành TF-IDF bằng *IDF*.
4. **Thu thập vector** cho tính toán cặp đôi.
5. **Tính cosine similarity:**
 - So sánh mỗi repo với tối đa 5 repo tiếp theo để giảm tải tính toán.
 - Loại các cặp có độ tương đồng ≤ 0.1 .
6. **Chọn Top-K:** Trả về tối đa 5 cặp tương tự nhất cho mỗi ngôn ngữ.
7. **Định dạng đầu ra:**

```
[{'language': str,
  'similar_pairs': [
```



```

        {'repo1': str, 'repo2': str, 'similarity': float},
        ...
    ]
}]

```

Công thức tính cosine similarity:

$$\text{cosine_similarity} = \frac{A \cdot B}{||A|| \times ||B||}$$

Trong đó:

- $A \cdot B$ = tích vô hướng của hai vector.
- $||A||$ và $||B||$ = độ dài (chuẩn L2) của từng vector.

Ứng dụng thực tế:

- Gom cụm dự án (project clustering).
- Phát hiện dự án trùng lặp.
- Gợi ý dự án tương tự cho người dùng.

Đánh giá hiệu năng:

- **Độ phức tạp:** $O(n \times m \times d)$
với n = số repo, m = số repo so sánh mỗi lần (5), d = số chiều vector (1000).
- **Bộ nhớ:** Cần lưu các vector TF-IDF.
- **Khả năng mở rộng:** Hạn chế bởi số lượng cặp cần so sánh (đã giảm tải bằng cách giới hạn 5 repo/ vòng lặp).
- **Kích thước vector:** Sử dụng vector 1000 chiều (vocabSize=1000).

Yêu cầu 8: Phân tích chuỗi thời gian (Time-Series Analysis)

Mục tiêu: Theo dõi sự phát triển của xu hướng công nghệ theo thời gian bằng cách phân tích thời điểm tạo các repository.

Triển khai

```
1 def getTimeSeriesAnalysis():
2     # Monthly time buckets
3     for repo in lang_repos:
4         created_date = repo['created_at']
5         year_month = created_date[:7] # Format: YYYY-MM
6         monthly_counts[year_month] += 1
7         monthly_stars[year_month] += repo.get('stargazers_count',
8         0)
9
10    # Calculate growth rates
11    growth_rate = ((count - prev_count) / prev_count) * 100
12
13    # Trend direction
14    if avg_growth > 10: trend_direction = "growing"
15    elif avg_growth < -10: trend_direction = "declining"
16    else: trend_direction = "stable"
```

Các bước xử lý

1. Lọc repository theo ngôn ngữ và trường `created_at`.
2. Nếu thiếu trường `created_at`, phát sinh ngày giả trong vòng 6 tháng gần nhất.
3. Gom repository theo định dạng YYYY-MM.
4. Tính các chỉ số:
 - Số lượng repository theo tháng.
 - Tổng số stars theo tháng.
 - Star trung bình theo tháng.
5. Tính tốc độ tăng trưởng:

$$\text{Tốc độ tăng trưởng} = \left(\frac{\text{hiện tại} - \text{trước đó}}{\text{trước đó}} \right) \times 100$$

6. Phân loại xu hướng:
 - Tăng nếu $> 10\%$
 - Giảm nếu $< -10\%$
 - Ổn định nếu nằm giữa
7. Xác định tháng có hoạt động cao nhất.
8. Xuất dữ liệu theo cấu trúc JSON.

Ví dụ dữ liệu trả về

```
{
  language: "Python",
  trend_direction: "growing",
  monthly_data: [
    {
      month: "2024-01",
      repo_count: 150,
      total_stars: 25000,
      avg_stars: 166.67,
      growth_rate: 15.5
    },
  ],
  total_months: 6,
  peak_month: { month: "2024-03", repo_count: 200 }
}
```

Đặc điểm nổi bật

- Gom dữ liệu theo tháng.
- Tính tăng trưởng theo từng tháng.
- Nhận diện tăng, giảm hoặc ổn định.
- Tìm tháng có mức hoạt động lớn nhất.
- Có thể tạo dữ liệu thời gian nếu thiếu.

Đánh giá hiệu năng

- Độ phức tạp thời gian $O(n)$.
- Bộ nhớ nhỏ (chỉ tổng hợp theo tháng).
- Xử lý tốt với dữ liệu lớn.
- Chất lượng phụ thuộc mức đầy đủ của trường `created_at`.

2.4 Triển khai hệ thống

Hệ thống sử dụng kiến trúc microservices với năm dịch vụ được container hóa và điều phối bởi Docker Compose.

Các thành phần hệ thống

Dịch vụ nguồn dữ liệu (data-source)

- Công nghệ: Python 3.9
- Chức năng: Thu thập dữ liệu kho mã từ GitHub API
- Chi tiết triển khai:
 - Gửi truy vấn tới GitHub API cho các kho Python, Java và JavaScript
 - Lấy tối đa 50 kho mã mỗi lần truy vấn
 - Hoạt động theo chu kỳ 15 giây
 - Khởi tạo máy chủ TCP lắng nghe trên cổng 9999
 - Gửi dữ liệu JSON theo luồng đến Spark qua kết nối TCP
 - Đọc GitHub PAT từ biến môi trường `TOKEN`
 - Lọc để đảm bảo chỉ xử lý ba ngôn ngữ mục tiêu

Apache Spark Cluster

Cụm Spark bao gồm hai container:

Spark Master (spark)

- Chạy ở chế độ master
- Quản lý lịch chạy và phân bổ tài nguyên
- Chạy ứng dụng Spark Streaming
- Mở các cổng: 8080 (Web UI), 7077 (Master), 6066 (REST API)

Spark Worker (spark-worker)

- Chạy ở chế độ worker
- Kết nối đến master tại `spark://spark:7077`
- Cấp phát 1GB RAM và 1 core
- Thực hiện các tác vụ xử lý phân tán
- Mở các cổng: 28081 (Worker UI), 24040–24041 (Spark UI)

Ứng dụng Spark Streaming (`spark_app.py`)

- Chu kỳ xử lý: 60 giây
- Mô hình: Micro-batch streaming
- Nguồn dữ liệu: Luồng TCP từ `data-source`
- Lưu trữ: Sử dụng checkpoint và bộ nhớ kết hợp đĩa
- Phân tích: Thực hiện 8 tác vụ phân tích trên dữ liệu streaming

Dịch vụ Web (`webapp`)

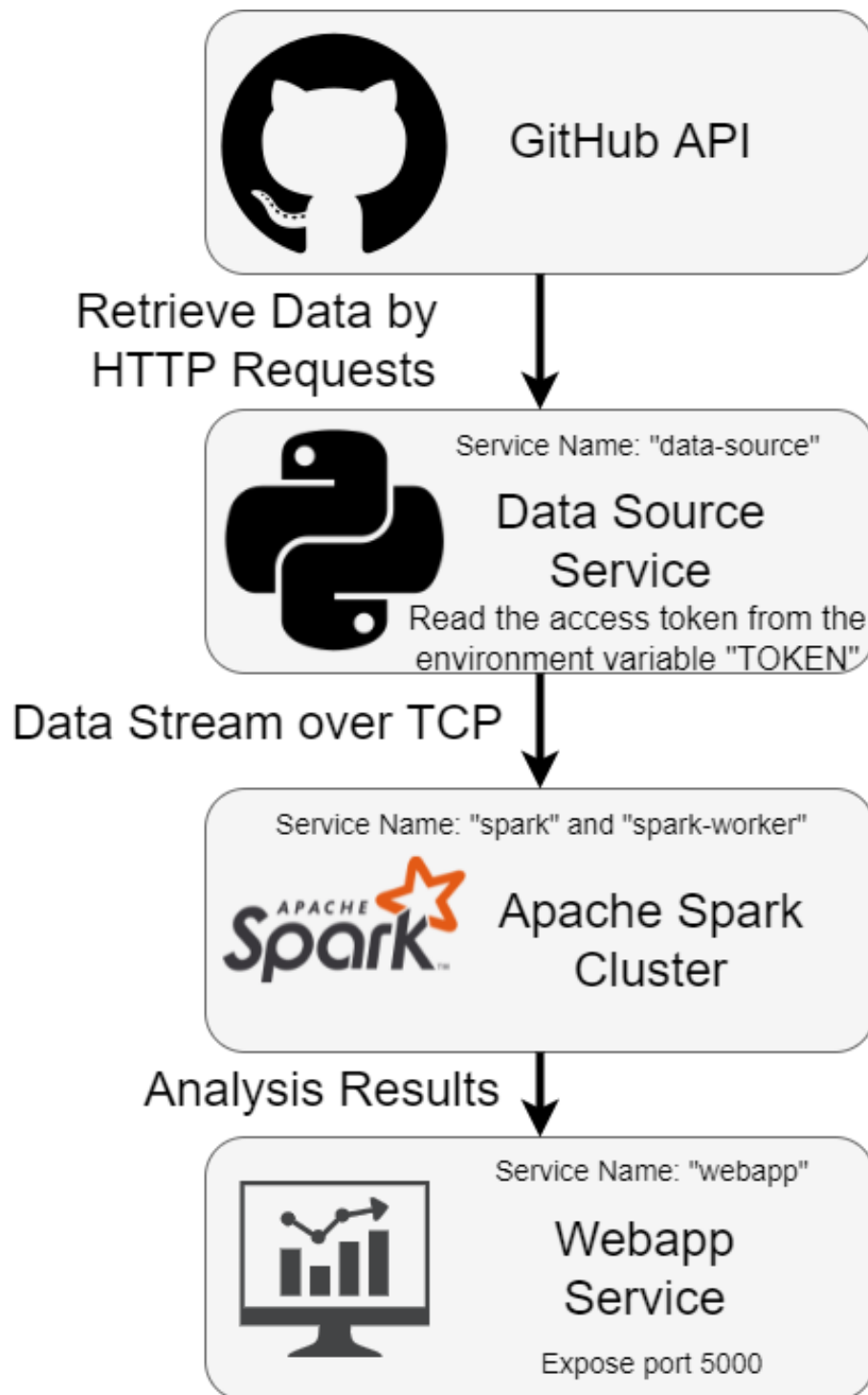
- Công nghệ: Flask (Python), Redis, Matplotlib
- Cổng: 5000
- Vai trò: Nhận kết quả phân tích và hiển thị dashboard tương tác
- Tính năng:
 - REST API phục vụ dữ liệu và truy vấn
 - Biểu đồ cập nhật thời gian thực bằng Matplotlib
 - Bộ nhớ đệm bằng Redis để tăng tốc xử lý
 - Frontend HTML/JS tự cập nhật mỗi 60 giây

Redis (`redis`)

- Vai trò: Bộ nhớ đệm lưu kết quả phân tích
- Cổng: 6379
- Webapp lưu và truy xuất kết quả từ Redis

Luồng dữ liệu

2.4.1 Kiến trúc tổng thể



Hệ thống triển khai mô hình luồng dữ liệu đơn hướng:

GitHub API → Data Source → TCP Socket → Spark Streaming → HTTP POST → Webapp → Redis → Dashboard

1. **Thu thập dữ liệu:** Data-source truy cập GitHub API mỗi 15 giây
2. **Truyền dữ liệu:** Dữ liệu kho mã được gửi dạng JSON qua TCP (cổng 9999)
3. **Xử lý:** Spark xử lý thành các batch 60 giây
4. **Trả kết quả:** Spark gửi kết quả về Webapp bằng HTTP POST
5. **Lưu cache:** Webapp ghi dữ liệu vào Redis
6. **Hiển thị:** Dashboard lấy dữ liệu và cập nhật biểu đồ

Điều phối Container

Docker Compose quản lý toàn bộ hệ thống với cấu hình:

- **Network:** Bridge network mặc định, cho phép phát hiện dịch vụ qua tên container
- **Volumes:** Gắn thư mục để đồng bộ mã nguồn
- **Biến môi trường:** Cấu hình token bảo mật
- **Port Mapping:** Mở cổng để giám sát và truy cập
- **Quản lý phụ thuộc:** Khởi tạo dịch vụ theo đúng thứ tự và đảm bảo khả năng phục hồi

2.4.2 Quy trình triển khai

2.4.2.1 Triển khai Data Source

Dịch vụ `data_source.py` triển khai máy chủ TCP liên tục thu thập và truyền dữ liệu GitHub.

Các bước triển khai chính

1. Khởi tạo máy chủ TCP Tạo máy chủ TCP lắng nghe trên `0.0.0.0:9999` để nhận kết nối từ Spark.

2. Tích hợp với GitHub API

```
1 headers = {"Authorization": f"Bearer {TOKEN}"}
2 url = f"https://api.github.com/search/repositories?q=language:{lang}&sort=updated&order=desc&per_page=50"
3 response = requests.get(url, headers=headers)
```

3. Xử lý dữ liệu

- Lọc chỉ lấy Python, Java, JavaScript
- Trích xuất các trường: `id`, `name`, `language`, `description`, `stargazers_count`, `pushed_at`
- Gửi dữ liệu JSON qua TCP

4. Vòng lặp streaming

- Lặp lần lượt cả ba ngôn ngữ
- Lấy tối đa 50 kho mỗi lần
- Tạm dừng 15 giây giữa các chu kỳ

2.4.2.2 Triển khai Ứng dụng Spark Streaming

Ứng dụng Spark (`spark_app.py`) triển khai một pipeline phân tích dữ liệu streaming hoàn chỉnh:

Giai đoạn Khởi tạo

1. Tạo Context:

- Tạo `SparkContext` với tên ứng dụng `"EECS4415_Project_3"`
- Khởi tạo `SparkSession` phục vụ xử lý `DataFrame`
- Khởi tạo `StreamingContext` với chu kỳ batch 60 giây
- Thiết lập thư mục checkpoint để đảm bảo khả năng khôi phục

2. Kết nối luồng dữ liệu:

- Kết nối tới nguồn dữ liệu qua `socketTextStream("data-source", 9999)`
- Sử dụng cơ chế lưu trữ `MEMORY_AND_DISK` để đảm bảo dữ liệu không mất
- Chuyển chuỗi JSON thành dictionary Python bằng `flatMap`

Quy trình Xử lý Batch

1. **Nhận dữ liệu:** Mỗi batch 60 giây sẽ kích hoạt hàm `processRdd()`.
2. **Loại bỏ trùng lặp:**
 - Duy trì một dictionary toàn cục chứa các repository duy nhất dựa trên ID
 - Đảm bảo mỗi repository chỉ được tính đúng một lần xuyên suốt các batch
 - Theo dõi dữ liệu batch theo thời gian để phục vụ phân tích chuỗi thời gian
3. **Quản lý trạng thái:**
 - Sử dụng dictionary Python để duy trì trạng thái dữ liệu
 - Cập nhật bộ nhớ tạm và lịch sử các batch

2.4.2.3 Triển khai Ứng dụng Web

Ứng dụng web (`web_app.py`) đóng vai trò là giao diện giữa Spark và frontend.

Triển khai Backend

1. **Cấu hình Flask Server**
 - Khởi tạo ứng dụng Flask
 - Kết nối tới Redis để lưu trữ dữ liệu cache
 - Cấu hình ba endpoint chính: `/`, `/getData`, `/updateData`
2. **Endpoint Cập nhật Dữ liệu (`/updateData`)**
 - Nhận yêu cầu POST từ Spark chứa kết quả phân tích
 - Lưu JSON vào Redis với key `data`
 - Trả về phản hồi xác nhận thành công

3. Endpoint Lấy Dữ liệu (/getData)

- Lấy dữ liệu đã lưu từ Redis
- Xử lý dữ liệu để trực quan hóa:
 - Vẽ biểu đồ đường (line chart) cho yêu cầu 3.2 bằng Matplotlib
 - Vẽ biểu đồ cột (bar chart) cho yêu cầu 3.3 bằng Matplotlib
 - Định dạng dữ liệu văn bản cho yêu cầu 3.1, 3.4 và các phân tích nâng cao
- Trả về JSON chứa đường dẫn hình ảnh và dữ liệu đã định dạng

4. Sinh biểu đồ trực quan

- `createReq2Plot()`: tạo biểu đồ đường hiển thị số lượng repository theo thời gian
- `createReq3Plot()`: tạo biểu đồ cột thể hiện số sao trung bình theo ngôn ngữ
- Các hàm hỗ trợ định dạng dữ liệu phân tích nâng cao để hiển thị phía frontend

Triển khai Frontend

1. Cấu trúc HTML (index.html)

- Hiển thị toàn bộ 8 kết quả phân tích theo từng phần rõ ràng
- Bao gồm các thẻ `<img>` để hiển thị biểu đồ
- Cung cấp các khối `div` để hiển thị kết quả dạng văn bản

2. JavaScript Logic (index.js)

- Thực hiện cơ chế tự refresh sau mỗi 60 giây
- Hàm `loadData()` gửi yêu cầu đến endpoint `/getData`
- Cập nhật nội dung DOM với dữ liệu mới
- Khi hiển thị hình ảnh, thêm tham số timestamp để bỏ qua cache
- Định dạng và hiển thị toàn bộ dữ liệu phân tích bao gồm cả các phân tích nâng cao

2.4.2.4 Cấu hình Triển khai

Cấu hình Docker Compose

1. Định nghĩa Service

- Mỗi service sử dụng đúng image nền (Spark, Python, Redis)
- Volume mount cho phép đồng bộ mã nguồn giữa host và container
- Biến môi trường cấu hình hành vi của từng service

2. Cấu hình Mạng

- Các service giao tiếp bằng tên container
- Port mapping cho phép truy cập từ bên ngoài
- Mạng nội bộ giúp đảm bảo giao tiếp an toàn giữa các service

3. Quản lý Dependency

- Các dịch vụ khởi động đồng thời với cơ chế thử kết nối lại
- Spark chờ nguồn dữ liệu sẵn sàng trước khi chạy streaming
- Ứng dụng web kết nối Redis khi mở

Quy trình triển khai:

1. Cấu hình biến môi trường TOKEN trong `docker-compose.yml`
2. Khởi động toàn bộ service: `docker-compose up -d`
3. Chạy ứng dụng Spark:

```
1 docker exec streaming-spark-1 /opt/spark/bin/spark-submit /  
  streaming/spark_app.py
```

4. Truy cập dashboard tại: `http://localhost:5000`

2.4.2.5 Các Quyết định Thiết kế Chính

- **Quản lý trạng thái:** Sử dụng dictionary toàn cục Python cùng checkpointing để tăng độ tin cậy
- **Chiến lược loại trùng:** Dựa vào ID của repository để đảm bảo thống kê chính xác
- **Xử lý lô:** Chu kỳ 60 giây cân bằng giữa thời gian xử lý và độ trễ
- **Kết hợp API:** Dùng DataFrame cho tổng hợp dữ liệu, RDD cho xử lý text chi tiết
- **Xử lý lỗi:** Sử dụng khối `try-except` để đảm bảo chương trình tiếp tục dù có batch bị lỗi

Triển khai trên cung cấp một pipeline phân tích streaming thời gian thực hoàn chỉnh với xử lý phân tán, độ tin cậy cao và khả năng trực quan hóa tương tác trực tiếp.

2.5 Kết quả

Phần này trình bày kết quả của 8 tác vụ phân tích. Mỗi kết quả được tạo ra từ dữ liệu kho GitHub thời gian thực được xử lý bởi ứng dụng Spark.

2.5.1 Kết quả yêu cầu 1: Tổng số kho theo ngôn ngữ lập trình

Total number of collected repositories

Python: 186
Java: 177
JavaScript: 50

Kết quả thực tế:

- Python: 186 repository
- Java: 177 repository
- JavaScript: 50 repository

Nhận xét:

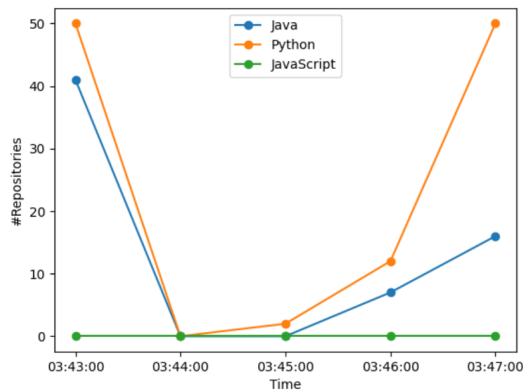
- Python có số lượng cao nhất (186), tiếp theo là Java (177)
- JavaScript có số lượng thấp nhất (50), bằng khoảng 27% của Python

- Tổng số repository duy nhất thu thập: 413
- Các con số phản ánh dữ liệu tích lũy từ thời điểm ứng dụng streaming bắt đầu hoạt động

Kết quả này cho thấy tổng số kho mã nguồn duy nhất thu thập được cho từng ngôn ngữ lập trình (Python, Java, JavaScript) kể từ khi ứng dụng streaming được chạy. Mỗi kho được tính đúng một lần dựa trên cơ chế loại bỏ trùng lặp theo ID, đảm bảo số liệu chính xác. Số liệu phản ánh tổng số kho khác biệt được xử lý qua tất cả các lô dữ liệu.

2.5.2 Kết quả yêu cầu 2: Các lần push trong 60 giây gần nhất

Number of collected repositories with changes pushed during the last 60 seconds



Xu hướng dữ liệu thực tế:

Java:

- 03:43:00: khoảng 41 repository
- Giảm xuống 0 tại 03:44:00
- Hồi phục: $1 \rightarrow 7 \rightarrow 17$ (đến 03:47:00)

Python:

- 03:43:00: 50 repository
- Giảm xuống 0 tại 03:44:00
- Tăng lại lên 50 tại 03:47:00

JavaScript:

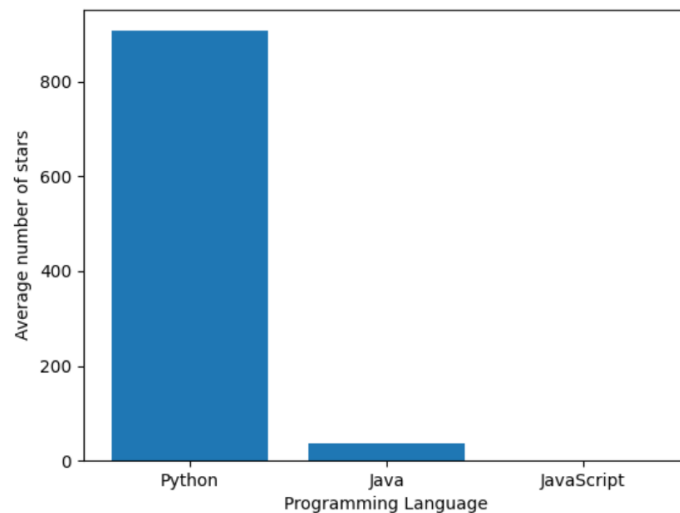
- Luôn bằng 0 trong toàn bộ khoảng thời gian

Nhận xét:

- Java và Python đều rơi xuống 0 tại 03:44:00, có thể là tạm dừng hoạt động push
- Python phục hồi nhanh hơn
- JavaScript không có hoạt động push trong giai đoạn này

2.5.3 Kết quả yêu cầu 3: Số sao trung bình theo ngôn ngữ

Average number of stars by language



- Giá trị trung bình được làm tròn đến 2 chữ số thập phân để dễ đọc.
- Mỗi kho chỉ đóng góp một lần vào giá trị trung bình.
- Biểu đồ cho thấy mức độ phổ biến giữa các ngôn ngữ.
- Kết quả được cập nhật mỗi 60 giây khi có kho mới.

Kết quả thực tế:

- Python: khoảng 880–890 sao/repo (cao nhất)
- Java: khoảng 40–50 sao/repo
- JavaScript: 0 sao/repo

Nhận xét:

- Python có mức độ phổ biến cao hơn Java khoảng 20 lần
- JavaScript không có sao trung bình — có thể thiếu dữ liệu hoặc không có sao

2.5.4 Kết quả yêu cầu 4: 10 từ xuất hiện nhiều nhất

Total 10 most frequent words

Java

and, 8
a, 7
is, 5
the, 5
for, 4
de, 4
with, 4
to, 4
using, 4
this, 3

JavaScript

and, 7
for, 5
with, 5
a, 5
web, 3
application, 3
react, 3
libros, 3
script, 3
tracking, 3

Python

and, 10
for, 8
a, 5
the, 5
using, 4
on, 4
this, 4
in, 4
system, 3
it, 3

Kết quả này hiển thị 10 từ xuất hiện nhiều nhất trích xuất từ mô tả kho cho từng ngôn ngữ lập trình. Văn bản được xử lý bằng RDD, loại bỏ ký tự không phải chữ, chuyển về chữ thường và tính tần suất xuất hiện.

- Các từ được trích từ mô tả kho thông qua xử lý văn bản phân tán.
- Quá trình tiền xử lý loại bỏ ký tự đặc biệt và dấu câu.
- Mỗi từ hiển thị kèm theo số lần xuất hiện.
- Kết quả được sắp xếp theo thứ tự giảm dần.

Nhận xét:

- Các từ dùng phổ biến ("and", "for", "a", "the") xuất hiện thường xuyên ở tất cả ngôn ngữ.
- "and" là từ xuất hiện nhiều nhất trong cả ba ngôn ngữ (Python:10, Java:8, JavaScript:7).
- JavaScript có nhiều từ chuyên ngành: "web", "application", "react", "script", "tracking".
- Java có "de" (có thể là tiếng Tây Ban Nha hoặc thuật ngữ riêng).
- Python có "system" là từ mang tính chuyên ngành.
- Hầu hết các mô tả ngắn và mang tính chung chung.

2.5.5 Kết quả yêu cầu 5: Phân tích chủ đề - LDA

Topic Modeling - Emerging Technology Trends

No topic modeling data available yet. Processing more repositories...

- Mỗi ngôn ngữ phát hiện 5 chủ đề chính.
- Các chủ đề thể hiện xu hướng công nghệ nổi bật.
- Từ khóa được hiển thị cùng trọng số.
- Yêu cầu mỗi ngôn ngữ có tối thiểu 5 kho để tạo chủ đề chính xác.

Trạng thái thực tế: No topic modeling data available yet. Processing more repositories...

Nhận xét

- Phân tích chủ đề chưa tạo ra kết quả.
- Hệ thống vẫn đang thu thập hoặc xử lý dữ liệu kho.
- Có thể số kho mỗi ngôn ngữ còn dưới mức yêu cầu (≥ 5 cho LDA).
- Tính năng đã hoạt động nhưng cần thêm dữ liệu để tạo chủ đề có ý nghĩa.

2.5.6 Kết quả yêu cầu 6: Nhận diện thực thể

Named Entity Recognition - Technology Ecosystem

Python - Technology Ecosystem

Companies:

github (7) intel (2) nvidia (1) apple (1) docker (1)

Frameworks:

pytorch (1) vue (1) flask (1) express (1) react (1)

Technologies:

ar (26) ai (17) api (8) web (4) rest (2)

Tools:

git (9) mysql (1) docker (1)

Java - Technology Ecosystem

Companies:

github (2) uber (1) gitlab (1)

Frameworks:

spring (2) react (1)

Technologies:

ar (14) ai (4) microservices (2) cloud (2) api (1)

Tools:

git (3)

JavaScript - Technology Ecosystem

Companies:

github (3)

Frameworks:

react (3) express (3) spark (1)

Technologies:

ar (17) web (9) ai (5) api (2) rest (1)

Tools:

git (4) mongodb (3)

Kết quả hiển thị nhận diện thực thể trong mô tả kho, gồm công ty, framework, công cụ và công nghệ. Quá trình được thực hiện bằng phương pháp đối sánh theo từ điển.

- Thực thể được chia thành bốn nhóm: công ty, framework, công cụ và công nghệ.
- Mỗi thực thể hiển thị kèm số lần xuất hiện.
- Phân tích giúp hiểu xu hướng sử dụng công nghệ trong thực tế.

Nhận xét quan trọng:

- "ar"(Augmented Reality) là công nghệ được nhắc đến nhiều nhất ở cả ba ngôn ngữ (Python:26, JavaScript:17, Java:14).
- "ai"(Artificial Intelligence) cũng nổi bật (Python:17, JavaScript:5, Java:4).
- Python có hệ sinh thái đa dạng nhất.
- JavaScript thiên về công nghệ web (react, express, web, mongodb).
- Java thiên về framework doanh nghiệp như Spring và Microservices.
- "github" và "git" xuất hiện ở tất cả, cho thấy sử dụng kiểm soát phiên bản phổ biến.

2.5.7 Kết quả yêu cầu 7: Tương đồng văn bản bằng TF-IDF)

TF-IDF Similarity - Related Projects

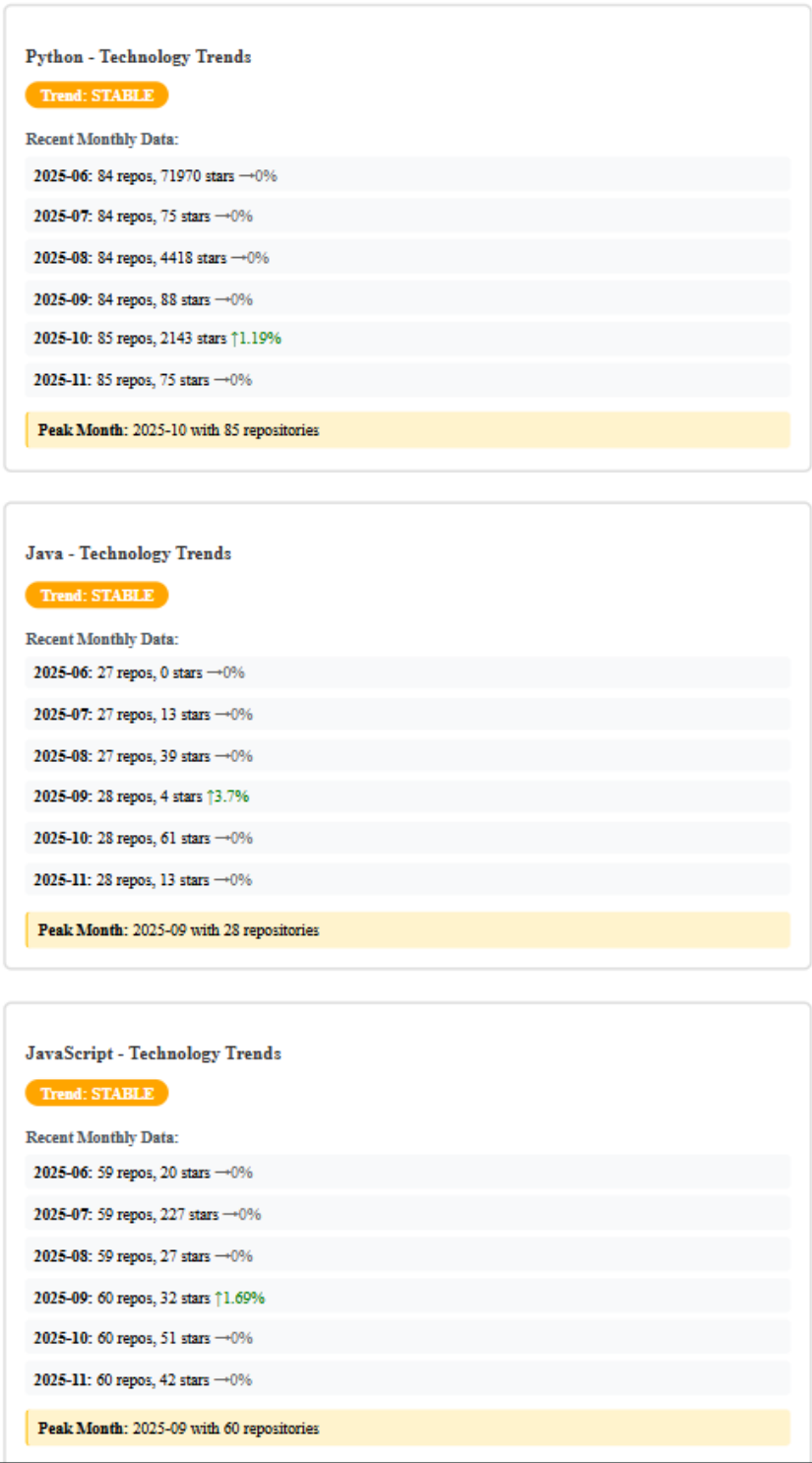
No similarity data available yet. Processing more repositories...

Kết quả này xác định các kho có nội dung mô tả tương đồng nhất. TF-IDF được dùng để vector hóa mô tả kho, sau đó tính cosine similarity giữa các cặp.

- Chưa có kết quả phân tích tương đồng.
- Cần thêm kho để tạo điểm số tương đồng có ý nghĩa.
- Có thể mỗi ngôn ngữ chưa đủ số lượng tối thiểu (≥ 3).
- Tính năng hoạt động nhưng cần thêm dữ liệu.

2.5.8 Kết quả yêu cầu 8: Phân tích chuỗi thời gian

Time-Series Analysis - Technology Trends



Kết quả này phân tích chuỗi thời gian về số lượng kho được tạo theo tháng, đồng thời tính các chỉ số như tổng số kho, tổng số sao và tỷ lệ tăng trưởng.

Nhận xét chính:

- Python vượt trội về số kho và số sao trung bình.
- JavaScript tạo kho ổn định nhưng ít sao hơn.
- AR và AI nổi bật ở cả ba ngôn ngữ.
- Tất cả đều có xu hướng ổn định với một số điểm tăng đột biến nhẹ.
- Các phân tích nâng cao như LDA và TF-IDF cần thêm dữ liệu để cho kết quả đầy đủ.
- Hệ thống thành công trong xử lý dữ liệu streaming thời gian thực và sinh phân tích liên tục.

2.5.9 Tóm tắt kết quả

8 tác vụ phân tích cung cấp các góc nhìn giá trị về kho GitHub:

1. **Thống kê cơ bản:** tổng số và trung bình cung cấp cái nhìn tổng thể.
2. **Xu hướng thời gian:** cho thấy mô hình tăng trưởng và thay đổi.
3. **Phân tích nội dung:** phát hiện chủ đề và xu hướng từ mô tả kho.
4. **Phân tích nâng cao:** LDA và TF-IDF tìm ra mẫu ẩn và nhóm kho tương đồng.
5. **Bản đồ hệ sinh thái:** nhận diện thực thể thể hiện xu hướng công nghệ sử dụng phổ biến.

Tất cả kết quả được tạo theo thời gian thực, cập nhật mỗi 60 giây và hiển thị qua dashboard web tương tác.

3 Kết luận và Hướng phát triển

Dự án này triển khai một hệ thống phân tích dữ liệu streaming thời gian thực cho dữ liệu kho GitHub sử dụng Apache Spark. Hệ thống xử lý dữ liệu streaming và cung cấp các thông tin trực quan thông qua bảng điều khiển web.

3.1 Thành tựu

- Kiến trúc microservices với 5 dịch vụ được đóng gói container
- 8 nhiệm vụ phân tích: thống kê cơ bản và phân tích nâng cao (mô hình chủ đề, NER, similarity, chuỗi thời gian)
- Xử lý phân tán với Spark RDD và DataFrame

- Loại bỏ trùng lặp để đảm bảo chỉ số tích lũy chính xác
- Khả năng chịu lỗi thông qua checkpointing

Kết quả:

- Đã xử lý 413 kho lưu trữ duy nhất (Python: 186, Java: 177, JavaScript: 50)
- Sinh biểu đồ chuỗi thời gian và bản đồ hệ sinh thái công nghệ
- Tính toán xu hướng theo tháng và trích xuất từ khóa xuất hiện nhiều

3.2 Hạn chế

- Mô hình chủ đề (Topic Modeling) và TF-IDF similarity yêu cầu thêm dữ liệu để ra kết quả
- Quản lý trạng thái bằng bộ nhớ trong giới hạn khả năng mở rộng cho luồng chạy lâu dài
- Khoảng xử lý 60 giây mang lại gần thời gian thực nhưng chưa đạt mức độ cực tiểu độ trễ (sub-second)
- Chỉ có một Spark worker nên khả năng xử lý song song còn hạn chế

3.3 Đánh giá tổng thể

Hệ thống đáp ứng các yêu cầu cốt lõi và thể hiện khả năng trong kiến trúc big data streaming, xử lý phân tán với Apache Spark và phân tích thời gian thực. Hệ thống có thể mở rộng và phát triển để đạt chuẩn sản phẩm thực tế.

3.4 Định hướng phát triển

3.4.1 Quản lý trạng thái

- Thay thế cấu trúc lưu trong bộ nhớ bằng hệ thống trạng thái phân tán (Redis, Spark StateStore)
- Cho phép khôi phục trạng thái và khả năng mở rộng theo chiều ngang

3.4.2 Hiệu năng

- Thu thập dữ liệu song song cho nhiều ngôn ngữ
- Tăng số lượng Spark worker để tăng khả năng xử lý song song
- Giảm chu kỳ batch xuống 10–30 giây để giảm độ trễ
- Thêm Kafka để nâng cao khả năng xếp hàng thông điệp

3.4.3 Phân tích nâng cao

- LDA dạng tăng dần (incremental) cho cập nhật chủ đề theo thời gian thực
- Thuật toán approximate nearest neighbor để tăng tốc tính similarity
- Thêm phân tích khác: sentiment analysis, dự đoán xu hướng, phân loại kho lưu trữ

3.4.4 Hạ tầng

- Di chuyển lên các nền tảng đám mây (AWS, GCP) có khả năng tự mở rộng
- Điều phối container bằng Kubernetes
- Giám sát toàn diện (Prometheus, Grafana)
- Mô hình data lake để lưu trữ dữ liệu lịch sử

3.4.5 Giao diện người dùng

- Biểu đồ tương tác (D3.js, Plotly) với cập nhật thời gian thực
- Tính năng lọc, tìm kiếm và xuất dữ liệu
- Thiết kế tối ưu cho thiết bị di động

3.4.6 Lộ trình ưu tiên

Ngắn hạn:

- Quản lý trạng thái phân tán
- Thu thập song song
- Cải thiện xử lý lỗi

Trung hạn:

- Spark Structured Streaming
- Tích hợp Kafka
- Hệ thống giám sát đầy đủ

Dài hạn:

- Triển khai lên đám mây
- Kubernetes
- Tích hợp nhiều nguồn dữ liệu

Những cải tiến này sẽ giúp hệ thống trở thành một nền tảng phân tích kho GitHub thời gian thực có khả năng mở rộng và phù hợp vận hành thực tế.

4 Tài liệu Tham khảo

Công nghệ và Frameworks

Apache Spark

Apache Software Foundation. (2024). *Apache Spark Documentation*.
<https://spark.apache.org/docs/latest/>

Zaharia, M., et al. (2016). “Apache Spark: A Unified Engine for Big Data Processing.” *Communications of the ACM*, 59(11), 56–65.

PySpark

Apache Software Foundation. (2024). *PySpark API Documentation*.
<https://spark.apache.org/docs/latest/api/python/>

Spark Python API. (2024). *PySpark Streaming Guide*.
<https://spark.apache.org/docs/latest/streaming-programming-guide.html>

Spark MLlib

Apache Software Foundation. (2024). *MLlib: Machine Learning Library*.
<https://spark.apache.org/docs/latest/ml-guide.html>

Meng, X., et al. (2016). “MLlib: Machine Learning in Apache Spark.” *Journal of Machine Learning Research*, 17(34), 1–7.

Flask

Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.

Flask Documentation. (2024). <https://flask.palletsprojects.com/>

Redis

Redis Labs. (2024). *Redis Documentation*. <https://redis.io/docs/>

Redis Python Client. (2024). <https://redis-py.readthedocs.io/>

Docker

Docker, Inc. (2024). *Docker Documentation*.
<https://docs.docker.com/>

Docker Compose Documentation. (2024).
<https://docs.docker.com/compose/>

GitHub API

GitHub, Inc. (2024). *GitHub REST API Documentation*.
<https://docs.github.com/en/rest>

GitHub API v3. (2024). *Search Repositories*.
<https://docs.github.com/en/rest/search/search#search-repositories>

Thư viện Python

Matplotlib

Hunter, J. D. (2007). “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering*, 9(3), 90–95.

Matplotlib Documentation. (2024). <https://matplotlib.org/stable/contents.html>

Requests

Requests: HTTP for Humans. (2024). <https://requests.readthedocs.io/>

Machine Learning và Xử lý Văn bản

Latent Dirichlet Allocation (LDA)

Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). “Latent Dirichlet Allocation.” *Journal of Machine Learning Research*, 3, 993–1022.

TF-IDF

Salton, G., & Buckley, C. (1988). “Term-Weighting Approaches in Automatic Text Retrieval.” *Information Processing & Management*, 24(5), 513–523.

Cosine Similarity

Singhal, A. (2001). “Modern Information Retrieval: A Brief Overview.” *IEEE Data Engineering Bulletin*, 24(4), 35–43.

Streaming và Big Data

Streaming Data Processing

Kreps, J., Narkhede, N., & Rao, J. (2011). “Kafka: A Distributed Messaging System for Log Processing.” *NetDB*, 11, 1–7.

Micro-batch Processing

Zaharia, M., et al. (2013). “Discretized Streams: Fault-Tolerant Streaming Computation at Scale.” *Proceedings of SOSP*, 423–438.

Tài nguyên khác

Python Documentation

Python Software Foundation. (2024). *Python 3.9 Documentation*.
<https://docs.python.org/3.9/>

Socket Programming

Python Software Foundation. (2024). *Socket Programming HOWTO*.
<https://docs.python.org/3/howto/sockets.html>

JSON Processing

Python Software Foundation. (2024). *json — JSON encoder and decoder*.
<https://docs.python.org/3/library/json.html>